Figure 1: The QtCreator welcome screen

Figure 2: The *New Project* dialogue box

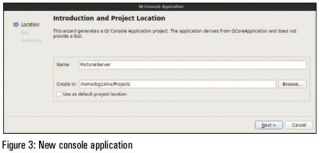

Figure 3: New console application

You're now ready to write code.

But before you do, remember that QtCreator projects are structured in a certain way. Every class has its own header file (which contains the class definition) and a *.cpp* file, which contains code written for the methods. You'll need to add classes using a wizard.

Hit *Ctrl+N* on the keyboard. The *New Project* dialogue box should come up. This time, there will be entries on the bottom-left section for *Files and Classes*. Select *C++* there, and then in the middle column, select *C++ Class*. Hit *Choose*.

In the wizard that comes up, give the class a name (I'm calling it FortuneServer again). Use the drop-down menu to select QObject as the base class (this is critical, because, as I learnt the hard way, if you don't do it here, QtCreator won't add that file to the list of files needed to be processed by the Meta-Object Compiler (MOC), and then make sure that the *Type Information* says *Inherits Qobject*.

Hit *Next*. Check the summary in the next screen, and hit *Finish* to create the new class. In the list of open files in the bottom left of QtCreator, you should see two more files—*fortuneserver.cpp* and *fortuneserver.h*. Now you're ready to fill it with code.

Figure 4: An empty project in QtCreator

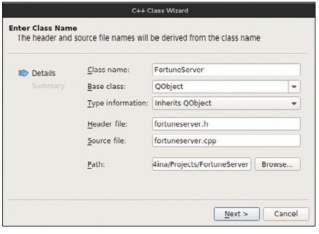

Figure 5: New class

Select *fortuneserver.h* in the *Open Documents* section to bring it into the editor. Let's start defining the class, as follows:

```
class FortuneServer : public QObject
{
    Q_OBJECT

    private:

    QTcpServer * server;
    QList<QByteArray> fortunes;

    public:

    explicit FortuneServer(QObject *parent = 0);
    ~FortuneServer();

    signals:

    private slots:

    void sendFortune();
};
```

This is all pretty standard stuff. The FortuneServer class inherits from QObject and has the *Q_OBJECT* macro, which sets it up for the MOC. We have a constructor (which needs to take the pointer to a QObject parent to set up the dependency tree) and a destructor. We also have a QList of QByteArrays, which stores all our fortunes. We also have a QTcpServer, which we'll use to set up the TCP server that handles all those connections that we'll get.

Notice that we didn't define any signals. That's because